# Lecture 1 Compendium: System Architecture & Convergence

AEC 813: Artificial Intelligence, IoT, and GIS Application in Agriculture

Dr. Kadeghe Fue
*PhD Agroecology*

Module 1: Introduction & Basics

## Contents

**1 Course Philosophy: The PhD Paradigm Shift** **2**

**2 The Internet of Things (IoT): Physics & Math** **2**
2.1 The Reference Architecture . . . 2
2.2 Deep Dive: Perception Layer (Sensor Physics) . . . 2
2.2.1 Mathematical Model for Calibration . . . 2
2.2.2 Python Simulation: Sensor Calibration . . . 3
2.3 Deep Dive: Network Layer (Link Budget) . . . 3
2.3.1 The Friis Transmission Equation . . . 3
2.3.2 Free Space Path Loss (FSPL) . . . 4
2.3.3 Python Calculation: IoT Range Estimation . . . 4

**3 Geographic Information Systems (GIS): The Context** **4**
3.1 Geodesy: The Shape of the Earth . . . 4
3.2 Distance Calculation: The Haversine Formula . . . 5

**4 Lab 1: The "Hello World" of Smart Farming** **5**
4.1 Objectives . . . 5
4.2 Implementation . . . 6

**5 Challenges in the Global South** **6**
5.1 Connectivity . . . 6
5.2 Power . . . 6
5.3 Cost . . . 7

**6 Review Questions & Critical Thinking** **7**

# 1 Course Philosophy: The PhD Paradigm Shift

Welcome to **AEC 813**. As PhD candidates, your primary objective shifts from *application* to *innovation*. While a Master's student might learn *how to fly* a drone to capture NDVI images, a PhD student must understand the *architectural limitations* of that drone's communication protocol, the *spectral physics* of the sensor, and the *mathematical validity* of the data processing pipeline.

This course is designed to equip you with the **"Full Stack AgTech"** capability. You will not view IoT, GIS, and AI as separate silos, but as a converged continuum of data:

1. **IoT (The Nerves):** The generation of data (Temporal resolution).
2. **GIS (The Context):** The contextualization of data (Spatial resolution).
3. **AI (The Brain):** The interpretation and actuation of data (Cognitive resolution).

**The Architect Mindset**

In developing countries like Tanzania, buying off-the-shelf solutions from Western markets often fails due to infrastructure gaps (power, connectivity, maintenance). Therefore, you must become **System Architects**. You will learn to design systems that are robust, repairable, and scientifically rigorous.

# 2 The Internet of Things (IoT): Physics & Math

The "Internet of Things" is a misnomer. It is better understood as the **Internet of Cyber-Physical Systems**. It bridges the gap between the analog world (soil, wind, light) and the digital world (bits, bytes, databases).

## 2.1 The Reference Architecture

We utilize a standard 4-Layer Architecture for agricultural IoT systems:

- **1. Perception Layer (Physical):** Sensors (Soil Moisture, NPK) and Actuators (Solenoid valves).
- **2. Network Layer (Transport):** Gateways, Protocols (LoRaWAN, MQTT, NB-IoT).
- **3. Middleware Layer (Processing):** Edge Computing, Cloud Platforms, Data Storage.
- **4. Application Layer:** User Interfaces, GIS Dashboards, Decision Support Systems (DSS).

## 2.2 Deep Dive: Perception Layer (Sensor Physics)

Sensors do not measure "Soil Moisture" directly; they measure proxies like **Voltage**, **Frequency**, or **Capacitance**. Rigorous PhD research requires calibration.

### 2.2.1 Mathematical Model for Calibration

Consider a capacitive soil moisture sensor. It measures the dielectric constant ($\epsilon$) of the soil-water matrix.

- Air (Dry) $\approx 1$
- Soil particles $\approx 3$–$5$
- Water $\approx 80$

The raw output is usually an analog voltage ($V_{out}$) converted to a digital integer (ADC value). We use a linear calibration model to convert this to Volumetric Water Content ($\theta_v$):

$$\theta_v = a \cdot V_{out} + b \tag{1}$$

Where:

- $\theta_v$ is Volumetric Water Content (%).
- $V_{out}$ is the raw sensor reading.
- $a$ (Slope) and $b$ (Intercept) are derived from gravimetric laboratory calibration.

### 2.2.2 Python Simulation: Sensor Calibration

The following Python code simulates raw ADC data and applies a calibration mapping.

```python
import numpy as np
import matplotlib.pyplot as plt

def simulate_sensor_calibration():
    # 1. Simulate Raw ADC Values (0-1023 for 10-bit ADC)
    # Assume Air (Dry) = 800, Water (Wet) = 400
    air_value = 800
    water_value = 400

    # Generate random raw readings representing a field
    raw_readings = np.random.randint(low=400, high=800, size=50)

    # 2. Apply Linear Calibration Map function
    # Maps input range [800, 400] to output range [0, 100]
    def map_values(x, in_min, in_max, out_min, out_max):
        return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min

    # Calculate moisture percentages
    moisture_percentages = map_values(raw_readings, air_value, water_value, 0,
    100)

    # 3. Output sample data
    print(f"Sample Raw ADC: {raw_readings[:5]}")
    print(f"Sample VWC (%): {np.round(moisture_percentages[:5], 2)}")

simulate_sensor_calibration()
```

Listing 1: Simulating Linear Sensor Calibration

## 2.3 Deep Dive: Network Layer (Link Budget)

A critical failure point in agricultural IoT is the wireless link. Will your sensor reach the gateway 5km away? This is determined by the **Link Budget**.

### 2.3.1 The Friis Transmission Equation

The received power ($P_{rx}$) must be greater than the receiver sensitivity ($S_{rx}$).

$$P_{rx} = P_{tx} + G_{tx} + G_{rx} - L_{path} - L_{misc} \tag{2}$$

Where:

- $P_{tx}$: Transmitter Power (dBm). (e.g., LoRa typical max = 20 dBm).

- $G_{tx}, G_{rx}$: Antenna Gains (dBi).
- $L_{path}$: Free Space Path Loss (FSPL).

### 2.3.2 Free Space Path Loss (FSPL)

For a signal frequency $f$ (MHz) and distance $d$ (km):

$$FSPL(dB) = 20\log_{10}(d) + 20\log_{10}(f) + 32.44 \tag{3}$$

### 2.3.3 Python Calculation: IoT Range Estimation

We calculate if a LoRaWAN sensor (868 MHz) can communicate over 5km.

```python
import math

def calculate_fspl(distance_km, frequency_mhz):
    # Constant 32.44 is for km and MHz units
    return 20 * math.log10(distance_km) + 20 * math.log10(frequency_mhz) + 32.44

def check_link_budget(distance_km, freq=868):
    # LoRaWAN Specifications
    tx_power = 20       # dBm (Max legal in many zones)
    tx_gain = 2         # dBi
    rx_gain = 3         # dBi
    sensitivity = -137  # dBm (LoRa SF12)

    loss = calculate_fspl(distance_km, freq)

    # Received Power Equation
    rx_power = tx_power + tx_gain + rx_gain - loss

    # Link Margin
    margin = rx_power - sensitivity

    print(f"--- Analysis @ {distance_km}km ---")
    print(f"Path Loss: {loss:.2f} dB")
    print(f"Received Power: {rx_power:.2f} dBm")
    print(f"Link Margin: {margin:.2f} dB")

    if margin > 0:
        print("RESULT: Connection Successful")
    else:
        print("RESULT: Connection Failed")

check_link_budget(5)
```

Listing 2: Link Budget Calculator

# 3 Geographic Information Systems (GIS): The Context

Without GIS, IoT data is just a number floating in a void. GIS anchors that number to a specific point on the Earth's surface, allowing for spatial correlation.

## 3.1 Geodesy: The Shape of the Earth

The Earth is an **Oblate Spheroid**. In your research, you must distinguish between:

- **GCS (Geographic Coordinate System):** Uses Lat/Long $(\phi, \lambda)$. Measured in Degrees. (e.g., WGS 84). *Do not use for area calculations.*

- **PCS (Projected Coordinate System):** Uses X/Y. Measured in Meters. (e.g., UTM Zone 37S for Morogoro).

### 3.2 Distance Calculation: The Haversine Formula

To calculate distance between two GPS points programmatically (without projecting), we use the Haversine formula:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cdot \cos\phi_2 \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \tag{4}$$

$$c = 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a}) \tag{5}$$

$$d = R \cdot c \tag{6}$$

Where $R$ is Earth's radius (approx. 6371 km).

```python
import math

def haversine_distance(lat1, lon1, lat2, lon2):
    R = 6371000  # Radius of Earth in meters

    # Convert degrees to radians
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlambda = math.radians(lon2 - lon1)

    a = math.sin(dphi/2)**2 + math.cos(phi1) * math.cos(phi2) * math.sin(dlambda/2)**2
    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1-a))

    return R * c

# Coordinates for Sokoine University of Agriculture (Approx)
sensor_A = (-6.848, 37.661)
sensor_B = (-6.852, 37.665)

dist = haversine_distance(sensor_A[0], sensor_A[1], sensor_B[0], sensor_B[1])
print(f"Distance between sensors: {dist:.2f} meters")
```

Listing 3: Haversine Distance Implementation

## 4 Lab 1: The "Hello World" of Smart Farming

In this practical module, we synthesize IoT and Data Analytics. We generate a mock dataset representing 3 soil moisture sensors in Morogoro and detect water stress events.

### 4.1 Objectives

1. Create a Pandas DataFrame with time-series data.
2. Implement a simple threshold-based algorithm.
3. Detect sensor failure (Anomaly Detection).

### 4.2 Implementation

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# --- STEP 1: GENERATE MOCK IOT DATA ---
# Create a date range (1 week, hourly data)
dates = pd.date_range(start='2024-01-01', periods=168, freq='H')

# Simulate moisture data for 3 sensors (0-100%)
np.random.seed(42)
# Sensor 1: Healthy field (Normal Dist, Mean 70)
s1_data = np.random.normal(70, 5, 168)
# Sensor 2: Drying field (Linear Decay)
s2_data = np.linspace(70, 20, 168) + np.random.normal(0, 2, 168)
# Sensor 3: Broken sensor (High Noise)
s3_data = np.random.normal(5, 50, 168)

df = pd.DataFrame({
    'Timestamp': dates,
    'Field_A': s1_data,
    'Field_B': s2_data,
    'Field_C': s3_data
})
df = df.set_index('Timestamp')

# --- STEP 2: ANALYZE DATA (Simple AI) ---
THRESHOLD = 30

# Detect periods where Field B is below threshold
alerts = df[df['Field_B'] < THRESHOLD]

print(f"ALERT: Field B required irrigation at {len(alerts)} time points.")
if len(alerts) > 0:
    print("First Alert Time:", alerts.index[0])

# Note: In a real scenario, this alert would trigger a solenoid valve via MQTT.
```

Listing 4: Smart Farming Data Analysis

## 5 Challenges in the Global South

As you design your PhD research, you must address the **Infrastructure Triad** specific to developing contexts.

### 5.1 Connectivity

**Problem:** 4G/5G is unreliable in rural areas.
**Solution:** Use **Store-and-Forward** architecture. Sensors buffer data locally on an SD card and transmit only when the network is available, or use LoRaWAN to bridge the gap to a town with fiber backhaul.

### 5.2 Power

**Problem:** Grid instability.
**Solution:** Solar harvesting is mandatory. Your link budget must account for "Sleep Cycles". A sensor that transmits every minute will drain a battery in days. A sensor transmitting every hour can last years.

### 5.3 Cost

**Problem:** Imported industrial sensors (e.g., Davis, Libelium) cost $500+.
**Solution:** Open Hardware (Arduino/ESP32). PhD research often involves calibrating low-cost capacitive sensors ($2) to match the accuracy of industrial TDR probes.

## 6 Review Questions & Critical Thinking

1. **Architecture:** If you place a soil sensor 10km from the gateway in a dense banana plantation, calculate the expected Path Loss at 868MHz. Will it work? If not, what parameters ($G_{tx}$, $P_{tx}$) can you change?
2. **Geodesy:** Why would calculating the area of a farm in Tanzania using WGS 84 (Lat/-Long) yield a different result than using UTM Zone 37S? Which one is legally robust for land tenure?
3. **Data Quality:** Refer to the "Field C" data in the python lab. Design a simple algorithm (pseudocode) to automatically detect that the sensor is broken (Hint: check for standard deviation or negative values).

## References

1. Kamal, R. (2017). *Internet of Things Architecture and Design Principles*. McGraw Hill.
2. Vuran, M. C., et al. (2018). *Internet of Underground Things in Agriculture: Architecture and Technology*. IEEE.
3. Ray, P. P. (2017). *Internet of things for smart agriculture: Technologies, practices and future direction*. Journal of Ambient Intelligence and Humanized Computing.